دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

درس هوش مصنوعی

گزارش پروژه اول: شبکه های عصبی بازگشتی برای تشخیص فعالیت انسانی

محمدامین حق‌جو ۴۰۳۱۱۰۵۸۵
پویا آسترکی ۴۰۳۱۱۰۴۷۷

مرداد ۱۴۰۵

چکیده

در این پروژه، ما به بررسی عملکرد مدل‌های متداول یادگیری ماشین برای مسائل ترتیبی و دارای حافظه می‌پردازیم و نقاط قوت و ضعف هر کدام از این ساختارها را در خصوص مسئله تشخیص فعالیت انسانی [1] بررسی می‌کنیم. ابتدا یک پیاده سازی پایه و ساده از شبکه‌های RNN را بررسی کرده و سپس معماری ساده، دو جهته و GRU از شبکهٔ LSTM را بررسی می کنیم.

کلیدواژه‌ها: شبکه‌های عصبی، یادگیری ماشین، تشخیص فعالیت انسانی، HAR، RNN، LSTM

[1] HAR

فهرست مطالب

۱ مقدمه و معرفی پروژه ۱

۱-۱ تعریف مسئله ۱

۱-۲ اهمیت موضوع ۱

۱-۳ ادبیات موضوع ۲

۲ مفاهیم اولیه ۳

۲-۱ نمادگذاری نظری ۳

۲-۱-۱ نماد های جبرخطی ۳

۲-۱-۲ نماد های یادگیری ماشین ۴

۲-۲ نماد گذاری کُد (عملی) ۵

۳ آماده سازی مجموعه دادگان ۶

۳-۱ خواندن مجموعه داده ۶

۳-۲ آماده سازی برای استفاده ۶

۴ پیاده سازی شبکه RNN از پایه ۷

۴-۱ روابط نظری ۷

۴-۲ نکات پیاده سازی عملی ۸

۴-۳ بررسی عددی صحت مقدار گرادیان ۸

۴-۴ نمودار کاهش خطای آزمایش ۹

۵ معماری آماده با PyTorch ۱۱

۶ بخش امتیازی: معماری GRU ۱۳

۷ نتایج و تحلیل ۱۴

۷-۱ مقایسه و تحلیل اندازه گرادیان ۱۴

۷-۲ آزمایش مقادیر مختلف T و H ۱۵

۷-۳ ارزیابی دقت مدل روی فعالیت‌های متفاوت ۱۵

۷-۴ مقایسه نتیجه مدل‌ها ۱۵

۷-۵ نتیجه بهترین مدل ۱۶

۷-۶ پاسخ به پرسش های تحلیلی ۱۷

مراجع ۱۸

فهرست تصاویر

۴-۱ بررسی عددی صحت محاسبه گرادیان ۹

۴-۲ نمودار کاهش خطای آموزش ۱۰

۶-۱ ماتریس درهم‌ریختگی مدل GRU ۱۳

۷-۱ نمودار اندازه گرادیان در طول زمان دو مدل ۱۴

۷-۲ ماتریس درهم‌رفتگی بهترین مدل ۱۶

فصل ۱

مقدمه و معرفی پروژه

در این فصل ابتدا به بیان مسئله پروژه، اهمیت آن و اهداف پروژه می پردازیم.

۱-۱ تعریف مسئله

تشخیص فعالیت حرکتی انسان‌ها به کمک شتاب‌سنج‌های موجود در ساعت‌ها و تلفن‌های هوشمند مسئله ای است که از نظر جامعه ورزشکاران و در حوزهٔ پزشکی اهمیت فراوانی دارد. به کار گیری روش‌های مدرن و دارای دقت بالای یادگیری ماشین در این حوزه برای این مقاصد اهمیت و مزایای زیادی به همراه دارد.

۱-۲ اهمیت موضوع

وجود یک مدل یادگیری ماشین دقیق برای تشخیص فعالیت انسانی، از جهات فراوانی اهمیت دارد، از جمله:

- بررسی فعالیت‌های روزانه یک ورزشکار یا بیمار بدون نیاز به تجهیزات پزشکی
- تسهیل فرآیند تشخیص فعالیت انسانی و صرفه‌جویی در هزینه تجهیزات پزشکی
- افزودن قابلیت اندازه‌گیری‌های طولانی فعالیت های یک فرد، بدون آنکه بر زندگی روزمرهٔ او تأثیر منفی‌ای بگذارد.

۱-۳ ادبیات موضوع

در این پروژه، ساختارهای متعددی از یادگیری ماشین به کار بسته ایم. شبکه های بازگشتی عصبی [۱]، شبکه های عصبی‌ای هستند که برای یادگیری توالی ترتیبی دنباله‌ها به کار می‌روند و برای این منظور، دارای دور های جهت دار درون شبکهٔ خود هستند. حافظهٔ بلندمدت کوتاه‌مدت [۲]، یک ساختار در یادگیری ماشین است که مشکلات موجود در شبکه های بازگشتی عصبی را حل می‌کند و می‌تواند وابستگی های ترتیبی دادگان را به خوبی یاد بگیرد.

---

[۱] RNN
[۲] LSTM

فصل ۲

مفاهیم اولیه

در این فصل عمدتاً به نمادگذاری به‌کار رفته در بخش نظری و عملی پروژه می‌پردازیم.

۲-۱ نمادگذاری نظری

۲-۱-۱ نماد های جبرخطی

در ابتدا، تمامی بردارها به شکل ستونی و با حروف کوچک انگلیسی و فونت پررنگ نوشته شده‌اند. مثال:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \ \mathbf{x} \in \mathbb{R}^n \tag{۱-۲}$$

ماتریس‌ها نیز با حروف بزرگی انگلیسی و فونت پررنگ نوشته شده‌اند. مثال:

$$\mathbf{y} = \mathbf{A}\mathbf{x} \tag{۲-۲}$$

مشتق کمیات برداری نیز با نمادگذاری ژاکوبی (نمادگذاری صورت) نوشته شده اند:

$$\frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial \mathbf{f}}{\partial x_1} & \frac{\partial \mathbf{f}}{\partial x_2} & \dots & \frac{\partial \mathbf{f}}{\partial x_n} \end{bmatrix} \tag{۳-۲}$$

$$\frac{\partial}{\partial \mathbf{x}}(\mathbf{A}\mathbf{x}) = \mathbf{A} \tag{۴-۲}$$

در نتیجهٔ این نمادگذاری، مشتق یک اسکالر نسبت به یک بردار، یک بردار سطری و ترانهادهٔ گرادیان آن است:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial x_1} & \frac{\partial \mathcal{L}}{\partial x_2} & \dots & \frac{\partial \mathcal{L}}{\partial x_n} \end{bmatrix} = \nabla \mathcal{L}^\top \tag{۵-۲}$$

۲-۱-۲ نماد های یادگیری ماشین

تابع ضرر[۱] را با نماد $\mathcal{L}$ نشان می‌دهیم. برای مجموعهٔ دادگان یادگیری ماشین، اندیس بالا نشان دهندهٔ شمارهٔ داده در مجموعهٔ دادگان است و اندیس پایین آن نشان دهندهٔ مؤلفه بردار است. به عنوان مثال، برای یک مجموعهٔ دادهٔ یادگیری $\mathcal{D} = \{(\mathbf{x}^i, y^i)\}_{i=1}^N$، که قرار است کلاس‌بندی روی C کلاس انجام دهد، تابع ضرر Cross Entropy چنین خواهد شد:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{C} y_c^i \log \hat{y}(\mathbf{x}^i)_c \tag{۶-۲}$$

توجه کنید که بردار $\mathbf{y}$، نمادگذاری One-Hot کلاس حقیقی y است، همچنین $\hat{\mathbf{y}}(\mathbf{x})$ نماد بردار احتمالات پیش‌بینی مدل برای ورودی $\mathbf{x}$ است.

تابع softmax می‌تواند به ازای ورودی، یک بردار یا یک ماتریس دریافت کند. ضابطهٔ این تابع برای هر دو حالت را اینجا ذکر می‌کنیم:

$$\text{softmax}(\mathbf{x}) = \begin{bmatrix} \frac{e^{x_1}}{\sum_{i=1}^n e^{x_i}} \\ \frac{e^{x_2}}{\sum_{i=1}^n e^{x_i}} \\ \vdots \\ \frac{e^{x_n}}{\sum_{i=1}^n e^{x_i}} \end{bmatrix} \tag{۷-۲}$$

$$\text{softmax}(\mathbf{A}) = \begin{bmatrix} \text{softmax}(\mathbf{a}_1) & \text{softmax}(\mathbf{a}_2) & \dots & \text{softmax}(\mathbf{a}_n) \end{bmatrix}, \ \mathbf{A} = \begin{bmatrix} \mathbf{a}_1 & \mathbf{a}_2 & \dots & \mathbf{a}_n \end{bmatrix} \tag{۸-۲}$$

[۱] Loss function

وقتی که یک تابع با دامنهٔ اعداد حقیقی را روی یک بردار اثر می‌دهیم، منظورمان این است که این تابع به تک‌تک مؤلفه‌های این بردار اعمال گردد و این حاصل برابر خروجی اثر تابع روی بردار شود. به عنوان مثال:

$$\tanh(\mathbf{x}) = \begin{bmatrix} \tanh(x_1) \\ \tanh(x_2) \\ \vdots \\ \tanh(x_n) \end{bmatrix} \tag{۹-۲}$$

۲-۲ نماد گذاری کُد (عملی)

به منظور استفاده آسان تر از کتابخانه های آماده جبرخطی در پایتون، ما بردارها را به صورت آرایه های دو بعدی numpy به شکل `(1, n)` ذخیره و استفاده می‌کنیم. در حقیقت انگار آنها را به شکل بردار سطری نگه داشته‌ایم.

```
import numpy as np

x = np.array([1, 2, 3]).reshape(1, 3)
```

در نتیجه این نمادگذاری، روابط ماتریسی ای که به صورت نظری نوشته بودیم، در کد دچار تغییر ترتیب ضرب یا ترانهاده می‌شوند، به عنوان مثال:

$$\mathbf{y} = \mathbf{A}\mathbf{x} \tag{۱۰-۲}$$

```
y = x @ A.T
```

فصل ۳

آماده سازی مجموعه دادگان

در این فصل، توضیحات مختصری درباره آماده سازی مجموعهٔ داده ارائه می‌کنیم.

۳-۱ خواندن مجموعه داده

به دلیل وجود تعداد محدودی اشتباه در نحوه قرارگیری داده‌ها، نیاز بود تا دقت بیشتری در خواندن مجموعه داده داشته باشیم. در بعضی از خطوط فایل داده‌های خام، بیش از یک داده نوشته شده است. همچنین، در برخی خطوط، خط به جای اتمام با کاراکتر ; ، با دو کاراکتر ; , تمام شده است. پس از رفع این مشکلات، ما اندازهٔ بردار شتاب را نیز در هر نقطه محاسبه و به مجموعهٔ داده اضافه کردیم.

۳-۲ آماده سازی برای استفاده

پس از خواندن داده های خام، آن‌ها را استاندارد کردیم و داده‌های مربوط به کاربران ۱ تا ۲۷ را در مجموعهٔ یادگیری، کاربران ۳۲ تا ۳۶ را برای مجموعهٔ تأیید و کاربران ۲۸ تا ۳۱ را برای مجموعهٔ آزمون قرار دادیم. این گونه هیچ اطلاعاتی از مجموعهٔ یادگیری به مجموعهٔ آزمون نشت نمی‌کند.

فصل ۴

پیاده سازی شبکه RNN از پایه

در این فصل، به توضیخات مربوط به پیاده سازی شبکهٔ RNN می‌پردازیم.

۴-۱ روابط نظری

برای گذر جلو[۱] در شبکه:

$$\mathbf{h}_t = \tanh(\mathbf{z}_t) \tag{۴-۱}$$

$$\mathbf{z}_t = \mathbf{W}_{xh}^\top \mathbf{x}_t + \mathbf{W}_{hh}^\top \mathbf{h}_{t-1} + \mathbf{b}_h \tag{۴-۲}$$

$$\mathbf{l} = \mathbf{W}_{hy}^\top \mathbf{h}_T + \mathbf{b}_y \tag{۴-۳}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{l}) \tag{۴-۴}$$

در اینجا، اندیس های زیر که متعلق به بردارها هستند، نشان دهنده اندیس آن‌ها در دنباله است. بدیهی است که داریم $\mathbf{h}_0 = \mathbf{0}$ برای محاسبه گرادیان تابع ضرر و گذر عکس[۲]، این روابط استفاده شدند:

[۱] Forward pass
[۲] Backwards pass

$$\frac{\partial \mathcal{L}}{\partial \mathbf{l}} = (\hat{\mathbf{y}} - \mathbf{y}_{\text{One-Hot}})^\top \tag{۴-۵}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_y} = \frac{\partial \mathcal{L}}{\partial \mathbf{l}} = (\hat{\mathbf{y}} - \mathbf{y}_{\text{One-Hot}})^\top \tag{۴-۶}$$

$$\frac{\partial \mathcal{L}}{\partial (W_{hy})_{ij}} = (h_T)_i (\frac{\partial \mathcal{L}}{\partial l_j}) \tag{۴-۷}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_T} = \frac{\partial \mathcal{L}}{\partial \mathbf{l}} \mathbf{W}_{hy}^\top \tag{۴-۸}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_t} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \odot (\mathbf{1} - \mathbf{h}_t \odot \mathbf{h}_t) \tag{۴-۹}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t-1}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_t} \mathbf{W}_{hh}^\top \tag{۴-۱۰}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_h} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \mathbf{z}_t} \tag{۴-۱۱}$$

$$\frac{\partial \mathcal{L}}{\partial (W_{xh})_{ij}} = \sum_{t=1}^{T} (x_t)_i (\frac{\partial \mathcal{L}}{\partial (z_t)_j}) \tag{۴-۱۲}$$

$$\frac{\partial \mathcal{L}}{\partial (W_{hh})_{ij}} = \sum_{t=1}^{T} (h_{t-1})_i (\frac{\partial \mathcal{L}}{\partial (z_t)_j}) \tag{۴-۱۳}$$

(۴-۱۴)

۴-۲ نکات پیاده سازی عملی

برای پیاده‌سازی شبکه، یک `class` تعریف کردیم و مِتُد های مورد نیازمان را آنجا نوشتیم. برای مقداردهی اولیه وزن‌ها و بایاس‌ها، از He initialization استفاده کردیم. برای پیاده‌سازی Gradient descent، شیوهٔ Small batch را انتخاب کردیم و در کد روی batch میانگین مشتق تابع ضرر را محاسبه می‌کنیم.

۴-۳ بررسی عددی صحت مقدار گرادیان

یک تابع مخصوص نوشته ایم تا با بسط تیلور مرتبهٔ اول و مشتق جهت‌دار، صحت گرادیان را تأیید کند:

$$\mathcal{L}(\boldsymbol{\theta} + \boldsymbol{\epsilon}) - \mathcal{L}(\boldsymbol{\theta} + \boldsymbol{\epsilon}) \approx 2 \frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}} \boldsymbol{\epsilon} \tag{۴-۱۵}$$

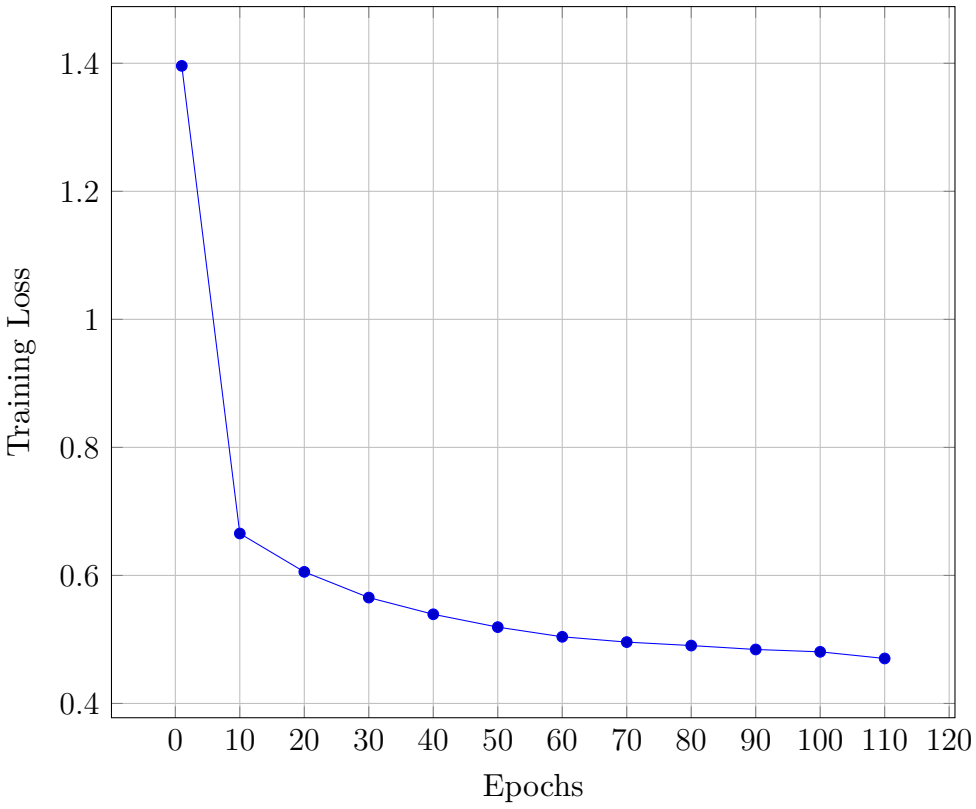
نتیجه این بررسی چنین است:

```
W_xh dir 1: num=1.144404e-01, ana=1.144404e-01, rel=6.083729e-10
W_xh dir 2: num=2.458762e-02, ana=2.458762e-02, rel=9.702931e-10
W_xh dir 3: num=2.431446e-02, ana=2.431446e-02, rel=9.317819e-10
W_hh dir 1: num=-1.470789e-01, ana=-1.470789e-01, rel=4.012097e-10
W_hh dir 2: num=-1.844954e-01, ana=-1.844954e-01, rel=1.814601e-10
W_hh dir 3: num=1.205656e-01, ana=1.205656e-01, rel=2.839666e-10
W_hy dir 1: num=6.733526e-02, ana=6.733526e-02, rel=1.880855e-10
W_hy dir 2: num=-1.456762e-01, ana=-1.456762e-01, rel=6.187571e-10
W_hy dir 3: num=-2.654421e-01, ana=-2.654421e-01, rel=8.527390e-11
b_h dir 1: num=-2.478101e-01, ana=-2.478101e-01, rel=2.101854e-10
b_h dir 2: num=7.461784e-01, ana=7.461784e-01, rel=4.001129e-12
b_h dir 3: num=2.694728e-01, ana=2.694728e-01, rel=3.489657e-10
b_y dir 1: num=4.968602e-01, ana=4.968602e-01, rel=3.187315e-12
b_y dir 2: num=-3.374631e-01, ana=-3.374631e-01, rel=7.270395e-11
b_y dir 3: num=-1.114543e-01, ana=-1.114543e-01, rel=3.491211e-10
W_xh: max relative error = 9.70e-10, max absolute error = 1.39e-10
W_hh: max relative error = 4.01e-10, max absolute error = 1.18e-10
W_hy: max relative error = 6.19e-10, max absolute error = 1.80e-10
b_h: max relative error = 3.49e-10, max absolute error = 1.88e-10
b_y: max relative error = 3.49e-10, max absolute error = 7.78e-11
```

شکل ۴-۱: بررسی عددی صحت محاسبه گرادیان

۴-۴ نمودار کاهش خطای آزمایش

برای مجموعهٔ وزن هایی که درنهایت انتخاب شدند، نمودار کاهش خطا روی مجموعهٔ داده‌های آموزش چنین است:



شکل ۴-۲: نمودار کاهش خطای آموزش

فصل ۵

معماری آماده با PyTorch

در بخش سه یک نوتبوک، داده‌هایی که از قبل آماده کرده‌بودیم را برای استفاده در محیط PyTorch به تانسور تبدیل کردیم تا بتوانیم با استفاده از ابزار دیتالودر آنها را به صورت دسته‌های کوچک‌تر به مدل بدهیم.

در بخش سه دو نوتبوک، برای چک کردن پیاده‌سازی خودمان خروجی مدل PyTorch را با خروجی مدل خودمان با وزن‌های یکسان مقایسه کردیم و به خطای ده به توان منفی هفت رسیدیم که مقدار قابل قبولی است.

در بخش سه سه نوتبوک، مدل‌های LSTM یک طرفه و دو طرفه را با استفاده از ماژول‌های آماده PyTorch پیاده‌سازی کردیم و برای جلوگیری از overfitting از dropout استفاده کردیم. برای آموزش مدل از بهینه‌سازی آدام استفاده کردیم که به صورت آماده از قبل موجود است و loss function ما هم Cross Entropy است و از L2 regularization هم برای جلوگیری از overfitting استفاده کردیم.

همچنین در epoch‌های ۱۵ و ۲۰ مقدار learning rate را کاهش می‌دهیم و همچنین early stopping هم داریم، به این شکل که اگر در ۵ epoch متوالی مقدار loss روی داده‌های ولیدیشن کاهش نیابد آموزش را متوقف می‌کنیم چون یادگیری بیشتر به احتمال زیاد بیهوده خواهد بود و غیر از اتلاف وقت می‌تواند باعث کاهش دقت شود چون عملا بهبودی در عملکرد مدل اتفاق نخواهد افتاد.

در بخش سه چهار نوتبوک، مدل را برای تعداد خیلی زیادی از هایپر پارامترها تست کردیم تا مقادیر معقولی را برای مدل نهایی مشخص کنیم و برخی نتایج را در فایل تکست ذخیره کردیم و نتیجه نهایی هم در output نوتبوک جوپیتر قرار دارد.

سپس در قسمت ارزیابی با استفاده از داده‌های تست عملکرد نهایی هر دو مدل را سنجیدیم و ماتریس

در هم ریختگی را برای تحلیل دقیق‌تر خروجی‌ها رسم کردیم تا مشخص شود مدل در تشخیص کدام فعالیت‌ها عملکرد بهتری داشته است.

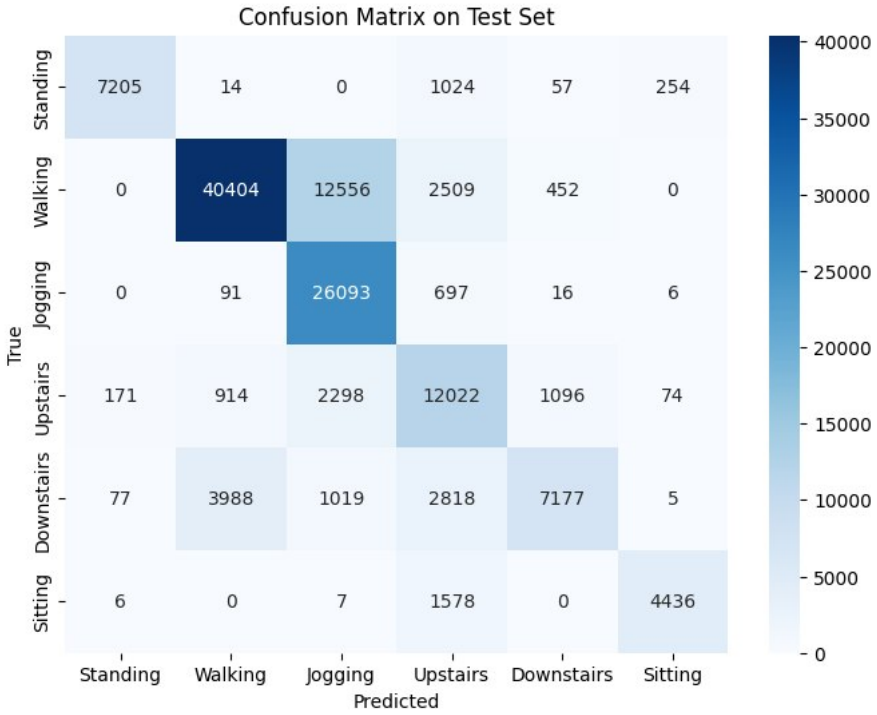
در قسمت بعدی مدل را روی داده‌های تست اجرا کردیم و confusion matrix و نتایج خواسته شده را خروجی دادیم. برای هر دو مدل، دقت‌های به نسبت خوبی گرفتیم که از مدل RNN عادی بهتر عمل کرد.

سپس نمودار اندازه گرادیان‌ها را برای مدل‌های LSTM و RNN ساده رسم کردیم که می‌بینیم مطابق انتظار کمینه اندازه گرادیان برای LSTM بسیار بزرگ‌تر از RNN ساده هست که در نتیجه هدف ما برای از بین بردن مشکل محو شدن گرادیان حل شده و تاثیر آن هم روی داده‌های تست دیدیم.

فصل ۶

بخش امتیازی: معماری GRU

ما برای بخش امتیازی معماری GRU را مورد بررسی قرار دادیم. دقت GRU خیلی با LSTM تفاوت معناداری نداشت، اما زمان یادگیری کوتاه‌تری دارد. ماتریس درهم‌ریختگی این مدل چنین ضمیمه گردیده‌است.



شکل ۶-۱: ماتریس درهم‌ریختگی مدل GRU

دقت مدل روی مجموعه آزمون نیز 75.42% است.

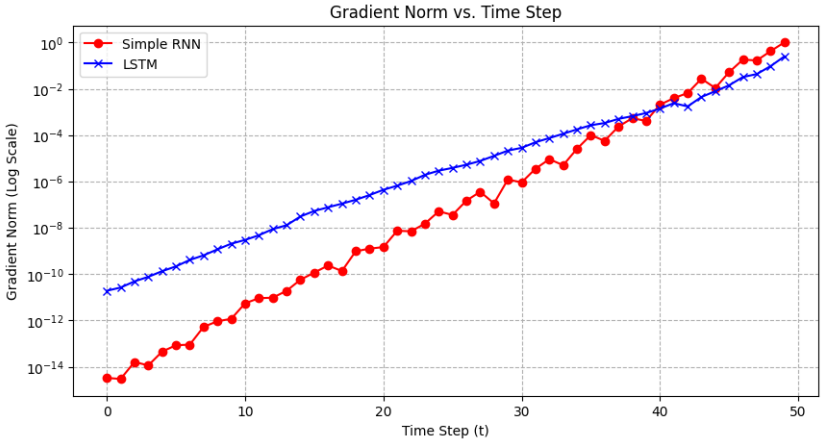
فصل ۷

نتایج و تحلیل

در این فصل، نتایج پروژه را تحلیل می‌کنیم.

۷-۱ مقایسه و تحلیل اندازه گرادیان

نمودار اندازه گرادیان در دو شبکهٔ ساده و LSTM چنین است:



شکل ۷-۱: نمودار اندازه گرادیان در طول زمان دو مدل

با توجه به نمودار مشخص است که گرادیان در معماری عادی خیلی بیشتر از LSTM افت می‌کند که همان مشکل محو شدن گرادیان است یعنی الگوهای طولانی مدت در LSTM بهتر یاد گرفته می‌شود و آپدیت هایی که برای وزن‌ها در LSTM صورت می‌گیرد محسوس‌تر و معنادارتر است در واقع این باعث می شود رابطه میان اعضای دورتر سری بهتر یاد گرفته شود.

۷-۲ آزمایش مقادیر مختلف T و H

با توجه به تست‌های متعددی که انجام دادیم، (که فقط تعداد محدودی از آن‌ها را ثبت نمودیم) با انتخاب مقادیر بزرگ برای T مدل به سرعت overfit می‌شود و همین‌طور برای مقادیر بزرگ لایه پنهان مثل ۶۴ هم همین اتفاق می‌افتد. با مقادیر کوچک هم underfitting داریم و نتیجه خوبی نمی‌گیریم. بهترین مقادیر پیدا شده هم در کد مشخص شده اند، که به اعداد ۲۵ برای T و ۳۲ برای H رسیدیم. نتایج تست در فایل `test_lstm.txt` ضمیمه شده‌اند.

۷-۳ ارزیابی دقت مدل روی فعالیت‌های متفاوت

label	precision	recall	f1-score	support
Standing	0.9657	0.7658	0.8542	8578
Walking	0.9335	0.7242	0.8156	55942
Jogging	0.6151	0.9144	0.7354	26908
Upstairs	0.5354	0.7352	0.6195	16580
Downstairs	0.7517	0.5657	0.6456	15090
Sitting	0.9312	0.7408	0.8251	6026

همان‌طور که مشاهده می‌کنیم، مدل ما در تشخیص و تمایز دو فعالیت Upstairs و Downstairs از یک‌دیگر، دارای مشکل است و دقت پایین‌تری دارد. دلیل این اتفاق، داشتن تنها یک لایهٔ پنهان در شبکه است، زیرا با داشتن تنها یک لایه، امکان استخراج ویژگی[۱] وجود ندارد، اگر می‌خواهیم مدل این دو فعالیت را بهتر از هم تشخیص دهد، بایستی حتماً ویژگی های تمایز دهنده را خودمان استخراج و به شبکه ورودی بدهیم.

۷-۴ مقایسه نتیجه مدل‌ها

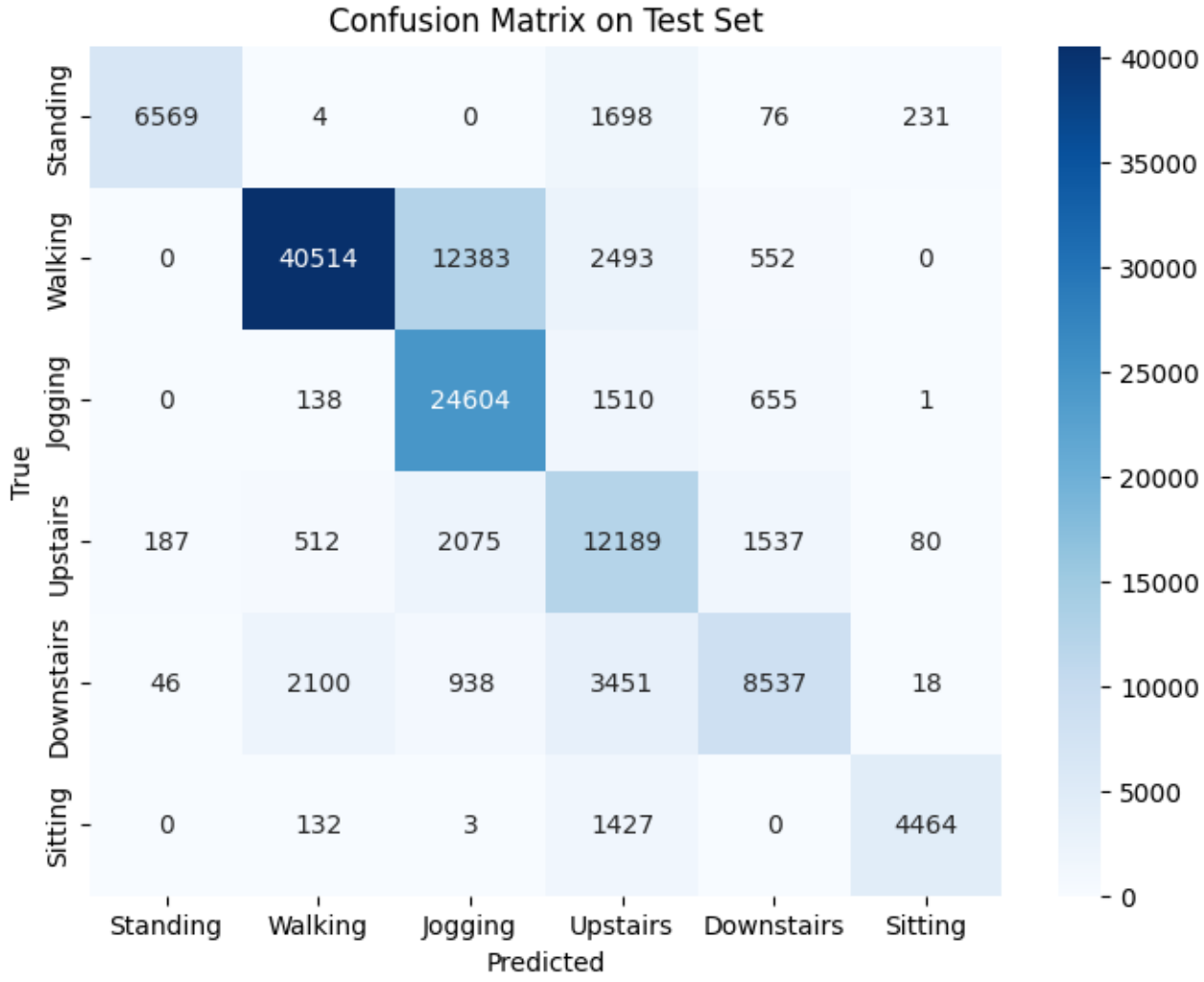
نتیجه و دقت هر مدل در جدول فوق ضمیمه شده است.

[۱] Feature extraction

زمان آموزش	macro-F1	دقت	مدل
48h	0.68	71.65%	شبکه‌ی ساده (از صفر)
36h	0.7493	75.03%	LSTM (PyTorch)
36h	0.7273	74.53%	Bidirectional LSTM

۷-۵ نتیجه بهترین مدل

ماتریس درهم‌رفتگی بهترین مدل که LSTM بود ضمیمه گردیده است.



شکل ۷-۲: ماتریس درهم‌رفتگی بهترین مدل

همان‌طور که از شکل ماتریس واضح است، فعالیت های Upstairs و Downstairs هستند که با هم اشتباه گرفته می‌شوند. همچنین، فعالیت Walking و Jogging هم تا حدی باهم اشتباه گرفته می‌شوند.

۶-۷ پاسخ به پرسش های تحلیلی

دقت در تقسیم تصادفی بیشتر است (انتظار داریم بیشتر باشد)چون بخشی از داده‌هایی که در یادگیری استفاده می‌شود در مرحله آزمون هم استفاده می‌شوند، که این باعث بالا رفتن دقت به صورت کاذب می‌شود اما تقسیم بر اساس کاربر این مشکل را حل می‌کند و جواب صادقانه‌تری می‌دهد که احتمالاً دقت کمتری دارد اما این دقت واقعی مدل هست.

شبکه های RNN عادی مشکل محوشدن گرادیان دارند که باعث می‌شود وقتی توی مرحله backpropagation مشتق های متعددی در یک‌دیگر ضرب شوند، وقتی که مقدار آنها زیر یک باشد، همین‌طور کوچک و کوچک‌تر می‌شوند که در نتیجه الگوهای بین اعداد خیلی قبل‌تر با عدد فعلی خوب یادگرفته نمی‌شود اما LSTM این مشکل را با اضافه کردن سلول حافظه و گیت‌ها حل کرده است. این ساختار مثل یک نوار عمل می‌کند که اجازه می‌دهد اطلاعات و گرادیان‌ها بدون این‌که ضرب بشوند و از بین بروند در طول زمان جریان پیدا کنند، به همین خاطر LSTM به راحتی می‌تواند وابسگی‌های طولانی تر هم یاد بگیرد.

مراجع

[1] Z. C. Lipton, J. Berkowitz, and C. Elkan. A critical review of recurrent neural networks for sequence learning, 2015.

[2] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[3] J. R. Kwapisz, G. M. Weiss, and S. Moore. Activity recognition using cell phone accelerometers. *SIGKDD Explor.*, 12(2), 2011.